

Artificial Neural Networks Model Design of Lorenz Chaotic System for EEG Pattern Recognition and Prediction

Lei Zhang¹

Abstract—This paper presents the preliminary work of a multidisciplinary brain research program. The goal of this research program is to generate accurate and effective signals for non-invasive brain stimulation, and deliver a hardware prototype to monitor and treat motion related mental disease such as Parkinson's and Epilepsy. It was shown in previous research that Electroencephalogram (EEG) signals captured from brain activities demonstrate chaotic features. Artificial neural network (ANN) resembles brain biological neural network and can be used to simulate chaotic system. The trained ANN model can in turn be used to analyze and control brain activities. In order to investigate the chaotic phenomenons of EEG signals and develop function for automatic pattern recognition, large amount of EEG signals are required. However, EEG signals are prone to noise and the available data is very limited. It is possible to control and predict the time series outputs of chaotic systems with known equations. Therefore, in order to study the dynamic control of the brain neural networks, an ANN architecture is designed and optimized for implementing Lorenz attractor to simulate the chaotic states of EEG signals. The research includes chaotic system, ANN design and the optimization of ANN architecture, which is based on the consideration of hardware implementation. The designed ANN model is trained with Lorenz attractor outputs with a fixed set of system parameters and the optimized architecture is selected based on the training results of three training algorithms and 16 ANN architectures with different number of hidden neurons.

I. INTRODUCTION

Previous research reported that brain activities captured by EEG demonstrate chaotic behaviors[1] [2]. Human brain cortex neurons have lateral connections with close neighbors to form behavior patterns from a network of neurons. Artificial neural network (ANN) architecture is inspired by biological neural network. Hence it can be used to model the chaotic phenomenons demonstrated by EEG signals captured from brain activities. The ANN model can aid the understanding of the brain dynamics and generate effective control signals for brain stimulation to treat neurological disease and improve brain functions.

The training of an ANN refers to generating weights and biases based on the inputs in order to minimize the error between the network outputs and the targeted outputs. A review of neural network can be found in[3]. Back-propagation (BP) is used for training multi-layer ANN to update weights and bias. The accuracy of the ANN relies on a large amount of training data. Therefore, there are two

main challenges in ANN design for the pattern recognition of Electroencephalogram (EEG) signals in brain research. 1) Limitation of available dataset. 2). Individual dependability. It is a time consuming task to collect large number of EEG sample signals due to the complexity of the process. Besides, the EEG signals are individually dependable and have to be preprocessed and normalized to be used as ANN inputs. It was reported that brain activities recorded EEG demonstrates chaotic behavior. The research goal is to explore the ANN topology design options and develop a general ANN design method for chaotic systems to represent the chaotic behaviors of brain activities for feature extraction and pattern recognition of EEG signals. The ANN is trained to learn the output patterns of Lorenz attractor.

The outputs of a chaotic system are bounded and follow a certain trajectory. Chaotic systems can be represented by a set of system equations, hence are predictable and controllable. However, the outputs of chaotic systems are extremely sensitive to initial conditions. Two identical chaotic systems with the same system parameters and just an infinitesimal offset of the initial values will start following the same trajectory, but quickly diverge into two completely different trajectories. A chaotic system can be stable, periodic or chaotic depending on its initial conditions and the system parameters. Lorenz attractor is a well studied chaotic system as represented by 3 ordinary differential equations (1), where the three system parameters are set as $\sigma = 10, \rho = 28, \beta = \frac{8}{3}$ to guarantee the outputs are chaotic. The outputs of Lorenz chaotic system are used for training the ANN.

$$\begin{aligned}\dot{x} &= \sigma(y - x) \\ \dot{y} &= \rho x - y - xz \\ \dot{z} &= -\beta z + xy\end{aligned}\tag{1}$$

ANN is often trained off-line on general computer and a fixed set of weights is obtained from the training. This training process can be time consuming. Conventional general-purpose processors are sequential and cannot truly support the parallelism of neural network architecture. Field programmable gate arrays (FPGA) has the advantages of parallelism and reconfigurability, and is suitable for emulating ANN architecture and accelerating the computation, making it possible to generate the weights and update the ANN architecture dynamically to adapt to new dataset. An FPGA implementation of ANN back propagation training algorithm is given by [4]. The hardware implementation of ANN requires dedicated FPGA logic resources for calculating multiplication. An optimized ANN architecture with

*The Ethic Approval is under review by the Research Ethic Board at the University of Regina. The presented work does not use any human data.

¹The author is with the Faculty of Engineering and Applied Science, University of Regina, Saskatchewan, Canada lei.zhang@uregina.ca

less number of hidden neurons can significantly reduce the FPGA hardware resource utilization, as well as improve the performance, namely, the operating speed.

II. ANN TOPOLOGY DESIGN

ANN topology is concerned with the number of hidden layers and the number of neurons in each layer. The ANN design aims to identify the optimal ANN architecture with simplest topology to meet the training target for chaotic systems. An ANN topology with excessive complexity would unnecessarily increase the computation load and consequently increase the hardware resource utilization and power consumption, as well as reduce the ANN training speed and hardware operating speed. It can also potentially cause overfitting of the training data and fail to generalize when apply the overfitted ANN model on new input data. On the other hand, an ANN with insufficient complexity will not generate the required chaotic patterns. The ANN design and topology optimization approach was used in previous research work for Hénon map[5] and Lorenz attractor[6] design; and can be easily extended to other chaotic systems.

The ANN is trained with MATLAB Neural Network Toolbox [7]. The ANN topology is defined by the number of hidden layers and the number of hidden neurons in each hidden layer. It is important to optimize the ANN topology in order to improve the FPGA hardware implementation performance, as it requires more hardware resources to implement more complex ANN topology. The ANN-based Lorenz system generator has a 3-layer architecture, with three neurons for x , y , z in both the input and output layers, and one hidden layer. The number of hidden neurons needs to be determined based on the training results in order to meet the training target with the minimal number of neurons and obtain the optimal ANN topology. Therefore, ANN with 1 to 16 hidden neurons are trained using three training algorithms respectively. Each training setup is repeated for 3 iterations to eliminate abnormal training results, as the initial values of weights and biases of the ANN are randomly assigned, which can cause the training performance to reach local minimum and stop the training process. There are a few commonly used ANN activation functions [8]. In the ANN design, the bipolar sigmoid activation function (*tansig*) is used for all hidden neurons and the ramp activation function is used for all output neurons.

In order to determine the optimum network topology, including the number of hidden layers and the number of neurons in each layer, it is necessary to go through a time-consuming training process for various topologies[5][6]. Generally, as the number of hidden layers or the number of neurons in the hidden layer increases, the processing speed of the ANN decreases. A good network topology should neither over-fit or under-fit the training data. A small network which under-fit the training data may misrepresent the target patterns, while a large network which over-fits the training data may not be able to generalize and thus have poor performance on the testing data. Moreover, a large network slows down the back propagation training process and is

undesirable for online training applications. It is important to choose the optimal ANN topology and numeric precision to improve the operating speed and reduce hardware resource utilization for the hardware implementation of ANN. High precision data format provides better accuracy and faster speed of convergence, but at the cost of more FPGA hardware resources.

III. TRAINING DATA AND TRAINING ALGORITHMS

An ANN can be trained by adjusting the weights to perform complex functions such as pattern recognition and classification. The ANN training process requires a dataset including many pairs of input and targeted output. These data pairs are used sequentially. After being given with one set inputs, the ANN output values are compared with the targeted output values and the errors are calculated and used to adjust the values of weights and bias, which will be applied to the next set of input/output data pair. This process carries on until the ANN outputs match the target within a predefined error margin.

The training data is generated using Lorenz system equations with the function *ode23* in MATLAB, which is equivalent to the forward Euler method. 10,000 training samples are generated, which are divided into three subsets: 70% of the samples are used for training; 15% are used for validation; and the rest 15% are used for testing. The best MSEs of the validation data set are used for evaluation. The ANN is actually trained in a similar way as for Recurrent Neural Network (RNN). The training process uses one time step of the chaotic system time series for the inputs and the next time step for the outputs. The ANN model is simulated with a feedback loop from the output to the input. The Nonlinear Input and Output (NIO) training model is used. The 10,000 samples are provided as inputs to the NIO model. The same data are used as target outputs but with one delay.

There are three training algorithms available from the MATLAB Neuron Network Toolbox: Levenberg-Marquardt (default algorithm), Scaled Conjugate Gradient algorithm (faster training speed) and Bayesian Regulation (slower but with better convergence). Each ANN architecture with different number of hidden neurons (1 to 16) is trained using all three training algorithms individually, repeated for three iterations. There are in total 192 training process (16 architectures x 3 training algorithms x 3 iterations). Each training iteration runs for 1000 epochs, where an epoch is when all the provided training samples are used for once to update the weights and biases. The values of the training parameters for three MATLAB training algorithms are set by default in MATLAB as previously reported in [6].

IV. TRAINING PERFORMANCE EVALUATION

The ANN training results are evaluated by the average Mean Squared Error (MSE) (2) between the trained ANN output y and the target training output $\hat{y}$, where N is the number of outputs, which is the total number of training samples (10,000) of all three time series (x , y and z) in the case of Lorenz attractor.



Fig. 1. ANN Training Performance for Lorenz Attractor

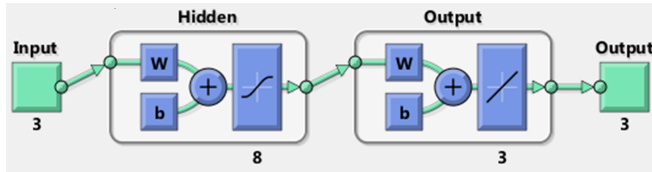


Fig. 2. ANN Architecture

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (2)$$

As the training data set is divided into three subsets: training, validation and testing, the training results of each subsets can be calculated independently. The training performance of the whole data set are calculated for comparing the training results of different training algorithms and ANN architectures.

The training performance of the Scaled Conjugate Gradient algorithm is shown in Fig 1(a). The MSEs increases generally while n increases. The best performance achieved with SCG training algorithm is $3.29E-3$ when $n=11$, while the training performances achieved by the LM and BR algorithms are significantly better, by up to 4 logarithmic

scale levels ($1E-2$ for SCG Verses $1E-6$ for both LM and BR) as indicated in Fig 1(d).

The training performance of the Bayesian Regulation algorithm is shown in Fig 1(b). The MSE values for all three iterations decrease below $1E-06$ when $n=9$. The performance improves negligibly as the number of the hidden neuron increases from $n=12$ for all three iterations. The overall trend is for the MSE to gradually decrease while the number of hidden neurons increases, but the effect for adding or removing one neuron for each training iteration is unpredictable, as shown by $n=6$ in iteration II.

The training performance of the Levenberg-Marguardt algorithm is shown in Fig 1(c). For all three iterations, the MSE does not consistently decrease while the number of hidden neurons increase. For examples, the MSEs of both iteration I and II increase when n is increased from 10 to 11; and the MSE of iteration III increases when n increases from 7 to 8. In general, the MSEs decreases with increased number of n . Although the overall performance is improving with increasing number of hidden neurons, it **cannot** be guaranteed that the performance can be improved by adding one neuron for any individual iteration. Hence it is necessary to train with multiple number of neurons numerically for multiple iterations. The best MSE of all three iteration for

LM decreases to the scale level of $1\text{E-}07$ when $n = 8$.

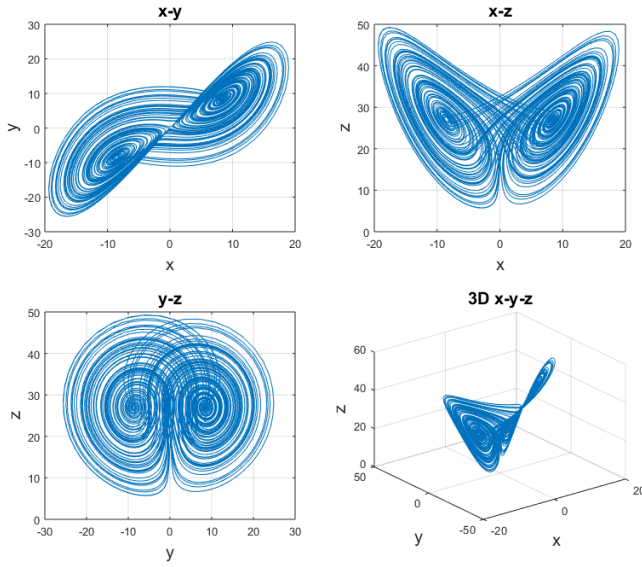


Fig. 3. ANN-based Lorenz Attractor Simulation

The average MSE values of all three iterations for each algorithm are compared in previous related research[9]. It is in fact more reasonable to compare the best performance achieved from the three training iterations, in order to eliminate the effect of abnormal iterations. The best performance of each algorithms are plotted in Fig 1(d). The training performance remains at the level of $1\text{e-}07$ with only insignificant improvement when the number of hidden neurons increase from 8 to 16 for all three training algorithms. Considering the computation cost of the ANN, the increased number of hidden neurons will increase the number of multiplications, which will in turn increase the resource utilization and potentially reduce the operating speed for hardware implementation. The weights and biases in the ANN model, as well as the training data are represented using 64-bit floating point data format in MATLAB simulation environment, while the FPGA hardware implementation uses fixed-point data format with quantization error limited by the number of bits used. The target MSE does not need to be smaller than the best resolution that can be represented by the selected fixed-point data format. For example, the smallest value that can be represented by 32-bit fixed-point data format with 16 fractional bits is $2^{-16} (\approx 1.526\text{E}-05)$, which is bigger than the smallest MSE achieved by the ANN model with 8 hidden neurons using the LM and BR training algorithms. Therefore, the ANN model is designed using LM training algorithm with 8 hidden neurons. The $3 \times 8 \times 3$ ANN architecture with 3 neurons in the input layer, 8 neurons in the hidden layer and 3 neurons in the output layer is illustrated by Fig.2 and the Simulink simulation of the ANN-based Lorenz attractor is shown in Fig 3. The total number of multiplications required for this ANN is 24 ($3 \times 8 + 8 \times 3$).

V. CONCLUSION & FUTURE WORK

This paper presented the ANN model design and topology optimization for implementing Lorenz chaotic system with a specific set of system parameters, which can be used to simulate and analyze the chaotic patterns of EEG signals, in order to advance the overall understanding of brain functionalities and the pathology of motion control related neurological diseases such as Parkinson's and epilepsy. The ANN is trained by 10,000 Lorenz chaotic system output samples, with 16 architectures using 1 to 16 neurons in the hidden layer respectively. Each architecture is trained with 3 training algorithms for 3 iterations. The optimal 3-layer ANN architecture has 3 input neurons, 8 hidden neurons and 3 output neurons. The ANN architecture optimization is important for improving the hardware performance of portable adaptive devices. The ANN Simulink model for Lorenz chaotic generator is created with the consideration of FPGA hardware implementation. The ANN model can be extended to adapt to system parameter changes, with system parameters as inputs. Future work includes ANN model design with various parameters of the Chaotic systems as inputs, and the hardware implementation of the chaotic generator on FPGA device with fixed-point data format in order to achieve optimized performance. The pattern recognition and prediction of chaotic behaviors can be carried out by calculating the Lyapunov exponent of the chaotic time series, and the dynamics of the chaotic system can be controlled by changing the input parameters.

REFERENCES

- [1] E. Pereda, R. Q. Quiroga, and J. Bhattacharya, "Nonlinear multivariate analysis of neurophysiological signals," *Progress in Neurobiology*, vol. 77, no. 1-2, pp. 1-37, sep 2005.
- [2] R. Falahian, M. Mehdizadeh Dastjerdi, M. Molaie, S. Jafari, and S. Gharibzadeh, "Artificial neural network-based modeling of brain response to flicker light," *Nonlinear Dynamics*, vol. 81, no. 4, pp. 1951-1967, 2015.
- [3] A. R. Omondi, J. C. Rajapakse, and M. Bajger, *FPGA Implementations of Neural Network*. Springer, 2006, ch. 1. FPGA Neurocomputers, pp. 1-36.
- [4] S. L. Pinjare and A. K. M., "Implementation of neural network back propagation training algorithm on fpga," *International journal of computer applications*, vol. 52, no. 1, pp. 1-7, Jan. 2012.
- [5] L. Zhang, "Artificial Neural Network Model Design and Topology Analysis for FPGA Implementation of Lorenz Chaotic Generator," in *2017 IEEE 30th Canadian Conference on Electrical and Computer Engineering (CCECE)*, Apr. 2017, pp. 216-219.
- [6] L. Zhang, "Design and Implementation of Neural Network Based Chaotic System Model for the Dynamical Control of Brain Stimulation," in *The Second International Conference on Neuroscience and Cognitive Brain Information (BRAININFO 2017)*, Nice, France, July 2017, pp. 14-21.
- [7] M. H. Beale, M. T. Hagan, and H. B. Demuth, *Neural Network Toolbox Reference*, r5015a ed., The MathWorks, Inc., Mar. 2015.
- [8] L. Zhang, "Implementation of Fixed-point Neuron Models with Threshold, Ramp and Sigmoid Activation Functions," in *4th International Conference on Mechanics and Mechatronics Research (ICMMR 2017)*, ser. IOP Conf. Series: Materials Science and Engineering, vol. 224, no. 012054, June 2017.
- [9] L. Zhang, "Hénon Map Chaotic System Analysis and VHDL-based Fixed-point FPGA Implementation for Brain Stimulation," in *2017 IEEE 30th Canadian Conference on Electrical and Computer Engineering (CCECE)*, Apr. 2017, pp. 808-811.